
Lab 03

Objective:

The objective of this lab is to learn how to implement perceptron – a basic building block of neural networks – in python.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement a perceptron model.
- Train perceptron using a dataset.

Instructor Note:

As pre-lab activity, read and implement the following tutorials:

1. <https://towardsdatascience.com/perceptron-and-its-implementation-in-python-f87d6c7aa428>
2. <https://machinelearningmastery.com/implement-perceptron-algorithm-scratch-python/>
3. <https://www.kaggle.com/code/kuzagaya/implementing-a-perceptron-in-python>

5) Useful Concepts

Perceptron: It is a system that learns using labeled examples (i.e., supervised learning) of feature vectors (or raw pixel intensities), mapping these inputs to their corresponding output class labels. In its simplest form, a Perceptron contains N input nodes, one for each entry in the input row of the design matrix, followed by only one layer in the network with just a single node in that layer (Figure 1).

There exist connections and their corresponding weights $w_1, w_2, \dots, w_i$ from the input x_i 's to the single output node in the network. This node takes the weighted sum of inputs and applies a step function to determine the output class label. The Perceptron outputs either a 0 or a 1-0 for class #1 and 1 for class #2; thus, in its original form, the Perceptron is simply a binary, two-class classifier.

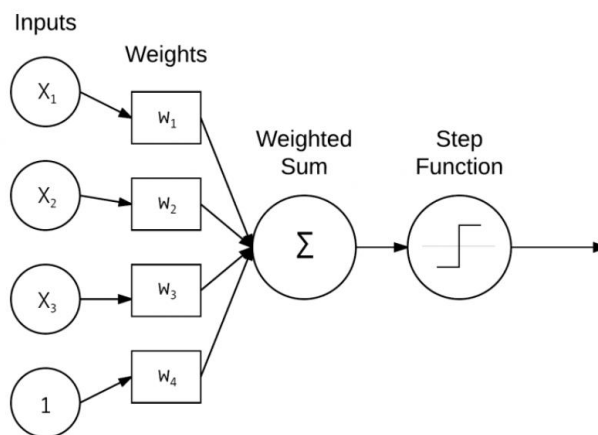


Figure 1: Architecture of perceptron.

Perceptron Training Procedure and the Delta Rule: Training a Perceptron is a fairly straightforward operation. Our goal is to obtain a set of weights $\mathbf{w}$ that accurately classifies each instance in our training set. In order to train our Perceptron, we iteratively feed the network with our training data *multiple times*. Each time the network has seen the *full set* of training data, we say an *epoch* has passed. It normally takes many epochs until a weight vector $\mathbf{w}$ can be learned to linearly separate our two classes of data. The pseudocode for the Perceptron training algorithm can be found below:

1. Initialize our weight vector $\mathbf{w}$ with small random values
2. Until Perceptron converges:
 - (a) Loop over each feature vector $\mathbf{x}_j$ and true class label d_j in our training set D
 - (b) Take $\mathbf{x}$ and pass it through the network, calculating the output value: $y_j = f(\mathbf{w}(\mathbf{t}) \cdot \mathbf{x}_j)$
 - (c) Update the weights $\mathbf{w}$: $w_i(t+1) = w_i(t) + \alpha(d_j - y_j)x_{j,i}$ for all features $0 \leq i \leq n$

Figure 2: Perceptron algorithm training procedure.

The actual “learning” takes place in Steps 2b and 2c. First, we pass the feature vector x_j through the network, take the dot product with the weights $\mathbf{w}$ and obtain the output y_j . This value is then passed through the step function which will return 1 if $x > 0$ and 0 otherwise.

Now we need to update our weight vector $\mathbf{w}$ to step in the direction that is “closer” to the correct classification. This update of the weight vector is handled by the *delta rule* in Step 2c.

The expression $(dj - yj)$ determines if the output classification is correct or not. If the classification is *correct*, then this difference will be zero. Otherwise, the difference will be either positive or negative, giving us the direction in which our weights will be updated (ultimately bringing us closer to the correct classification). We then multiply $(dj - yj)$ by x_j , moving us closer to the correct classification.

The value α is our *learning rate* and controls how large (or small) of a step we take. It’s *critical* that this value is set correctly. A larger value of α will cause us to take a step in the right direction; however, this step could be *too large*, and we could easily overstep a local/global optimum.

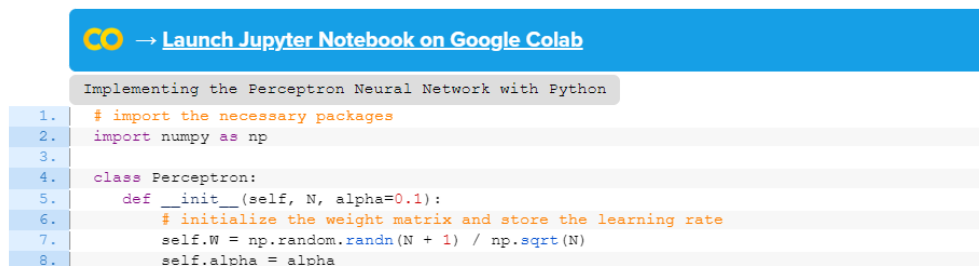
Conversely, a small value of α allows us to take tiny baby steps in the right direction, ensuring we don’t overstep a local/global minimum; however, these tiny baby steps may take an intractable amount of time for our learning to converge.

Finally, we add in the previous weight vector at time t , $\mathbf{w}_j(t)$ which completes the process of “stepping” towards the correct classification.

Understanding the Implementation of Perceptron in Python:

Now that we have studied the Perceptron algorithm, let’s implement the actual algorithm in Python.

Step 1: Create a file named `perceptron.py` in your jupyter notebook. After you’ve created the file, open it, and insert the following code:



```
CO → Launch Jupyter Notebook on Google Colab

Implementing the Perceptron Neural Network with Python

1. # import the necessary packages
2. import numpy as np
3.
4. class Perceptron:
5.     def __init__(self, N, alpha=0.1):
6.         # initialize the weight matrix and store the learning rate
7.         self.W = np.random.randn(N + 1) / np.sqrt(N)
8.         self.alpha = alpha
```

Line 5 defines the constructor to our Perceptron class, which accepts a single required parameter followed by a second optional one:

- N : The number of columns in our input feature vectors. In the context of our bitwise datasets, we'll set

N equal to two since there are two inputs.

- α : Our learning rate for the Perceptron algorithm. We'll set this value to 0.1 by default. Common choices of learning rates are normally in the range $\alpha = 0.1, 0.01, 0.001$.

Line 7 files our weight matrix W with random values sampled from a “normal” (Gaussian) distribution with zero mean and unit variance. The weight matrix will have $N + 1$ entries, one for each of the N inputs in the feature vector, plus one for the bias. We divide W by the square-root of the number of inputs, a common technique used to scale our weight matrix, leading to faster convergence. We will cover weight initialization techniques later in this chapter.

Step 2: Next, let's define the step function.

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```

10.     def step(self, x):
11.         # apply the step function
12.         return 1 if x > 0 else 0

```

This function mimics the behavior of the step equation — if x is positive we return 1, otherwise, we return 0.

Step 3: To actually train the Perceptron we'll define a function named `fit`. If you have any previous experience with machine learning, Python, and the scikit-learn library then you'll know that it's common to name your training procedure function `fit`, as in “fit a model to the data”:

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```

14.     def fit(self, X, y, epochs=10):
15.         # insert a column of 1's as the last entry in the feature
16.         # matrix -- this little trick allows us to treat the bias
17.         # as a trainable parameter within the weight matrix
18.         X = np.c_[X, np.ones((X.shape[0]))]

```

The `fit` method requires two parameters followed by a single optional one: The X value is our actual training data. The y variable is our target output class labels (i.e., what our network should be predicting). Finally, we supply `epochs`, the number of epochs our Perceptron will train for.

Line 18 applies the bias trick by inserting a column of ones into the training data, which allows us to treat the bias as a trainable parameter directly inside the weight matrix.

Step 4: Let's review the actual training procedure:

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```
20.         # loop over the desired number of epochs
21.     for epoch in np.arange(0, epochs):
22.         # loop over each individual data point
23.         for (x, target) in zip(X, y):
24.             # take the dot product between the input features
25.             # and the weight matrix, then pass this value
26.             # through the step function to obtain the prediction
27.             p = self.step(np.dot(x, self.W))
28.
29.             # only perform a weight update if our prediction
30.             # does not match the target
31.             if p != target:
32.                 # determine the error
33.                 error = p - target
34.
35.                 # update the weight matrix
36.                 self.W += -self.alpha * error * x
```

On Line 21, we start looping over the desired number of epochs. For each epoch, we also loop over each individual data point x and output target class label (Line 23). Line 27 takes the dot product between the input features x and the weight matrix W , then passes the output through the step function to obtain the prediction by the Perceptron. Applying the same training procedure detailed in Figure 3, we only perform a weight update if our prediction does not match the target (Line 31). If this is the case, we determine the error (Line 33) by computing the sign (either positive or negative) via the difference operation. Updating the weight matrix is handled on Line 36 where we take a step towards the correct classification, scaling this step by our learning rate α . Over a series of epochs, our Perceptron is able to learn patterns in the underlying data and shift the values of the weight matrix such that we correctly classify our input samples x .

Step 5: Lets understand the implementation of predict function.

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```
38.     def predict(self, X, addBias=True):
39.         # ensure our input is a matrix
40.         X = np.atleast_2d(X)
41.
42.         # check to see if the bias column should be added
43.         if addBias:
44.             # insert a column of 1's as the last entry in the feature
45.             # matrix (bias)
46.             X = np.c_[X, np.ones((X.shape[0]))]
47.
48.         # take the dot product between the input features and the
49.         # weight matrix, then pass the value through the step
50.         # function
51.         return self.step(np.dot(X, self.W))
```

Our predict method requires a set of input data X that needs to be classified. A check on Line 43 is made to see if a bias column needs to be added. Obtaining the output predictions for X is the same as the training procedure — simply take the dot product between the input features X and our weight matrix W , and then pass the value through our step function. The output of the step function is returned to the calling function. Now that we've implemented our Perceptron class, let's try to apply it to our bitwise datasets and see how the neural network performs.

Evaluating the Perceptron Bitwise Datasets: To start, let's create a file named `perceptron_or.py` that attempts to fit a Perceptron model to the bitwise OR dataset:

→ Launch Jupyter Notebook on Google Colab

Implementing the Perceptron Neural Network with Python

```
1. # import the necessary packages
2. from pyimagesearch.nn import Perceptron
3. import numpy as np
4.
5. # construct the OR dataset
6. X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
7. y = np.array([[0], [1], [1], [1]])
8.
9. # define our perceptron and train it
10. print("[INFO] training perceptron...")
11. p = Perceptron(X.shape[1], alpha=0.1)
12. p.fit(X, y, epochs=20)
```

Lines 2 and 3 import our required Python packages. We'll be using our Perceptron implementation. Lines 6 and 7 define the OR dataset based on Table 1. Lines 11 and 12 train our Perceptron with a learning rate of $\alpha = 0.1$ for a total of 20 epochs. We can then evaluate our Perceptron on the data to validate that it did, in fact, learn the OR function:

→ Launch Jupyter Notebook on Google Colab

Implementing the Perceptron Neural Network with Python

```
14. # now that our perceptron is trained we can evaluate it
15. print("[INFO] testing perceptron...")
16.
17. # now that our network is trained, loop over the data points
18. for (x, target) in zip(X, y):
19.     # make a prediction on the data point and display the result
20.     # to our console
21.     pred = p.predict(x)
22.     print("[INFO] data={}, ground-truth={}, pred={}".format(
23.         x, target[0], pred))
```

On Line 18, we loop over each of the data points in the OR dataset. For each of these data points, we pass it through the network and obtain the prediction (Line 21). Finally, Lines 22 and 23 display the input data point, the ground-truth label, as well as our predicted label to our console. To see if our Perceptron algorithm is able to learn the OR function, just execute the following command:

→ Launch Jupyter Notebook on Google Colab

Implementing the Perceptron Neural Network with Python

```
1. $ python perceptron_or.py
2. [INFO] training perceptron...
3. [INFO] testing perceptron...
4. [INFO] data=[0 0], ground-truth=0, pred=0
5. [INFO] data=[0 1], ground-truth=1, pred=1
6. [INFO] data=[1 0], ground-truth=1, pred=1
7. [INFO] data=[1 1], ground-truth=1, pred=1
```

Again, our Perceptron was able to correctly model the function. The AND function is only true when both $x_0 = 1$ and $x_1 = 1$ – for all other combinations the bitwise AND is zero. Finally, let's take a look at the nonlinearly separable XOR function inside `perceptron_xor.py`

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```

1. # import the necessary packages
2. from pyimagesearch.nn import Perceptron
3. import numpy as np
4.
5. # construct the XOR dataset
6. x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
7. y = np.array([[0], [1], [1], [0]])
8.
9. # define our perceptron and train it
10. print("[INFO] training perceptron...")
11. p = Perceptron(X.shape[1], alpha=0.1)
12. p.fit(X, y, epochs=20)
13.
14. # now that our perceptron is trained we can evaluate it
15. print("[INFO] testing perceptron...")
16.
17. # now that our network is trained, loop over the data points
18. for (x, target) in zip(X, y):
19.     # make a prediction on the data point and display the result
20.     # to our console
21.     pred = p.predict(x)
22.     print("[INFO] data={}, ground-truth={}, pred={}".format(
23.         x, target[0], pred))

```

Again, the only lines of code that have been changed are Lines 6 and 7 where we define the XOR data. The XOR operator is true *if and only if* one (but not both) x 's are one. Executing the following command we can see that the Perceptron *cannot* learn this nonlinear relationship:

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```

1. $ python perceptron_xor.py
2. [INFO] training perceptron...
3. [INFO] testing perceptron...
4. [INFO] data=[0 0], ground-truth=0, pred=1
5. [INFO] data=[0 1], ground-truth=1, pred=1
6. [INFO] data=[1 0], ground-truth=1, pred=0
7. [INFO] data=[1 1], ground-truth=0, pred=0

```

6) Solved Lab Activities

Sr.No	Allocated Time	Level of Complexity	CLO Mapping
1	25	Low	CLO-2
2	25	Low	CLO-2

Activity 1:

Load dataset: The dataset that we consider for implementing Perceptron is the Iris flower dataset. This dataset contains 4 features that describe the flower and classify them as belonging to one of the 3 classes. We strip the last 50 rows of the dataset that belongs to the class 'Iris-virginica' and use only 2 classes 'Iris-setosa' and 'Iris-versicolor' because these classes are linearly separable and the algorithm converges to a local minimum by eventually finding the optimal weights.

Solution:

```
import numpy as np
```

```
import pandas as pd

import matplotlib.pyplot as plt

def load_data():

    URL_='https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data'

    data = pd.read_csv(URL_, header = None)

    print(data)

    # make the dataset linearly separable

    data = data[:100]

    data[4] = np.where(data.iloc[:, -1]=='Iris-setosa', 0, 1)

    data = np.asmatrix(data, dtype = 'float64')

    return data

data = load_data()
```

Visualize the dataset: Visualizing the dataset with 2 of the features, we can see that that dataset can be clearly separated by drawing a straight line between them. Our goal is to write an algorithm that finds that line and classifies all of these data points correctly.

Solution:

```
plt.scatter(np.array(data[:50,0]), np.array(data[:50,2]), marker='o', label='setosa')

plt.scatter(np.array(data[50:,0]), np.array(data[50:,2]), marker='x', label='versicolor')

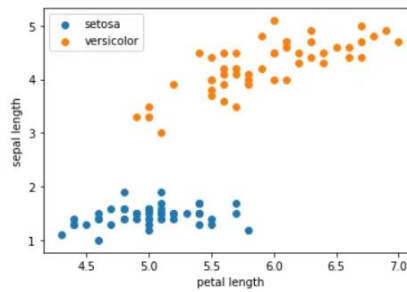
plt.xlabel('petal length')

plt.ylabel('sepal length')

plt.legend()

plt.show()
```

Output



Activity 2:

Implementation of perceptron

```
def perceptron(data, num_iter):  
    features = data[:, :-1]  
    labels = data[:, -1]  
    # set weights to zero  
    w = np.zeros(shape=(1, features.shape[1]+1))  
    misclassified_ = []  
  
    for epoch in range(num_iter):  
        misclassified = 0  
        for x, label in zip(features, labels):  
            x = np.insert(x,0,1)  
            y = np.dot(w, x.transpose())  
            target = 1.0 if (y > 0) else 0.0  
  
            delta = (label.item(0,0) - target)  
  
            if(delta): # misclassified  
                misclassified += 1  
                w += (delta * x)
```

```
        misclassified_.append(misclassified)

    return (w, misclassified_)

num_iter = 10

w, misclassified_ = perceptron(data, num_iter)
```

A property of the Perceptron is that if the dataset is linearly separable, then the algorithm is guaranteed to converge at some point!

```
epochs = np.arange(1, num_iter+1)

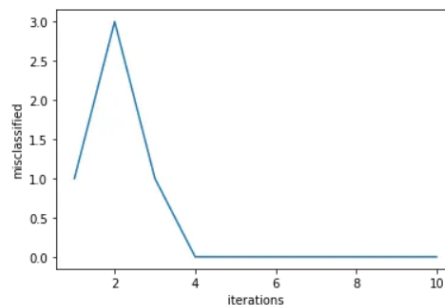
plt.plot(epochs, misclassified_)

plt.xlabel('iterations')

plt.ylabel('misclassified')

plt.show()
```

Output



7) Graded Lab Tasks

The purpose of this lab task is to help students understand, implement and train perceptron in Python.

Lab Task 1: Loading dataset: Load a dataset from Kaggle or UCI Machine Learning repository and visualize it.

Lab Task 2: Implement perceptron on the dataset, training it using training algorithm and show the learning curve and weights.